

Rapport: Samler, Historie, Finans, I min samling og UI/UX 8.6-brytere

1. Formål

Denne rapporten oppsummerer avklaringer, Neon-kontroller og låste regler fra chatten om hvordan Collectium UI/UX 8.6 skal bruke bryterne:

- Samler
- Historie
- Finans
- I min samling

Rapporten gjelder spesielt objektpresentasjon, visningskort, relasjonskoblinger, relasjonspresentasjon, katalog, periodefilter og videre kobling mot Neon/API.

Hovedkonklusjonen er at Collectium nå har et tydeligere skille mellom:

1. Katalogobjektet
2. Brukerens konkrete samlerobjekt
3. Objektets historiske relasjoner
4. Objektets finansielle markedsdata
5. Brukerens private samlingsdata

Dette skillet er viktig for at UI/UX 8.6 ikke skal blande katalogsannhet, historisk sannhet, markedsdata og private brukerdata.

2. Overordnet UI/UX 8.6-prinsipp

Samler, Historie, Finans og I min samling skal ikke være tilfeldige faner i frontend. De skal være styrte visningsbrytere basert på Neon/API-data.

Frontend skal ikke finne opp sannhet, felt, relasjoner, priser eller URL-er. Frontend skal vise det API og resolved views returnerer.

Det betyr:

- Visningskort skal bruke segmentene for kort oppsummering.
 - Objektpresentasjon skal bruke segmentene for full objektvisning.
 - Relasjonspresentasjon skal bruke segmentene for node-/relasjonsvisning.
 - Index og periodefilter skal kunne bruke samme relasjons- og segmentlogikk.
 - I min samling skal kun vise private brukerdata når bruker er innlogget og har rett tilgang.
-

3. Definisjon av de fire bryterne

3.1 Samler

Samler er katalogisk og samlerrelevant visning av objektet.

Samler skal svare på:

- Hva er objektet?
- Hvilken type objekt er det?
- Hvilken valør, utgave, variant og litra har det?
- Hvor sjeldent er objektet?
- Hvilken kvalitet eller katalogvurdering finnes?
- Kan brukeren lagre objektet, markere det eller legge det i samling?

Samler skal vise:

- Tittel
- Katalognummer
- Kilde
- Objekttype
- Valør
- År
- Publiseringsår
- Litra
- Valørutgave / serie
- Variant
- Signatur/person
- Regent
- Sjeldenhet
- Katalogvurdering
- Hjerter / ønskeliste
- Stjerne / favoritt
- Legg i samling
- Brukerstatus når bruker er innlogget

Samler skal hente fra:

- `ct_v_object_presentation_resolved`
- `ct_v_no_banknote_object_presentation`
- `ct_v_object_user_state_resolved` for brukerstatus

Samler skal ikke inneholde fulle private samlingsdata. Det kan vise små statusfelt som:

- I min samling
- Hjerter
- Stjerne
- Følges

Men kjøpspris, kjøpt fra, kvittering, egne bilder og private notater hører primært til I min samling.

3.2 Historie

Historie er objektets historiske og relasjonelle kontekst.

Historie skal svare på:

- Hvilken historisk periode tilhører objektet?
- Hvilken regent/konge er objektet knyttet til?
- Hvilke personer eller signaturer er koblet til objektet?
- Hvilken utgave, valør og variant er objektet del av?
- Hvilke andre objekter deler samme år, regent, person, valør, utgave eller variant?
- Hvilke relasjonssider kan åpnes?

Historie skal vise:

- År
- Publiseringsår
- Kilde
- Person/signatur
- Regent/konge
- Utgave
- Valør
- Variant
- Hovedperiode
- Utgaveperiode
- Historisk periode når data finnes
- Historiske hendelser når data finnes
- Proveniens/funn når data finnes
- Relasjonschips / relasjonslenker

Historie skal hente fra:

- `ct_v_object_presentation_resolved.history_json`
- `ct_v_no_banknote_object_presentation`
- `ct_v_object_relations_resolved`

Viktig: `ct_v_object_relations_resolved` er et koblingslag, ikke hele historiekilden. Den viser hvilke relasjoner objektet har og hvor brukeren skal sendes videre. Selve relasjonssiden må hente dypere data fra egne relation detail views.

Eksempel på dypere relasjonsskilder som senere må brukes:

- Regent-/konge-view
- Person-/signatur-view
- Utgave-/serie-view
- Valør-view
- Variant-view
- Periode-view
- År-/publiseringsår-view
- Global/europeisk/norsk historisk kontekst
- Funn/proveniens-view

- Finanshistorisk index-view
-

3.3 Finans

Finans er objektets markeds-, verdi- og økonomivisning.

Finans skal svare på:

- Har objektet markedsverdi?
- Finnes trend?
- Finnes prisobservasjoner?
- Finnes auksjonsstatus?
- Finnes nettbutikkstatus?
- Finnes grade-/kvalitetsverdier?
- Hvordan utvikler verdien seg over tid?
- Hvordan kan objektet sammenlignes med indeks, gull, sølv, valuta, kjøpekraft eller historisk økonomi?

Finans skal vise når data finnes:

- Estimert markedsverdi
- Verdi
- Trend
- Trendprosent
- Trendperiode
- Likviditet
- Siste prisobservasjon
- Auksjonsstatus
- Nettbutikkstatus
- Grade values / verdi per kvalitet
- Markedsgrunnlag
- Sammenligning med index
- Sammenligning med historisk økonomi

Finans skal hente fra:

- `ct_v_object_market_resolved`
- `finance_json`
- markeds-/prisobservasjonstabeller når disse finnes
- auksjon-/nettbutikkdata når disse finnes
- index-/økonomidata når disse finnes

Viktig låst regel:

Null eller manglende verdi skal aldri vises som reell `0 kr`.

Hvis data mangler, skal UI vise:

- Mangler markedsverdi
- Ikke beregnet trend

- Ikke registrert på auksjon
- Ikke registrert i nettbutikk
- Ingen grade-verdier registrert

Dette er viktig fordi `0 kr` kan tolkes som reell markedsverdi, mens `null` betyr manglende eller ikke beregnet data.

3.4 I min samling

I min samling er brukerens private visning av eget konkrete eksemplar.

I min samling skal svare på:

- Eier brukeren dette objektet?
- Når ble det kjøpt?
- Hvor ble det kjøpt?
- Hvem ble det kjøpt fra?
- Hva betalte brukeren?
- Hvilken kvalitet mener brukeren objektet har?
- Har brukeren egne bilder, kvitteringer eller notater?
- Er objektet privat, delt eller synlig?
- Har objektet privat proveniens?

I min samling skal vise:

- I min samling-status
- Hjerte
- Stjerne
- Følges
- Kjøpsdato
- Kjøpssted
- Kjøpt fra
- Kjøpspris
- Valuta
- Kvalitet ved kjøp
- Min kvalitet
- Gradering
- Plassering
- Synlighet
- Egne notater
- Kvittering
- Egne bilder
- Forside scan
- Bakside scan
- Privat proveniens
- Egne tagger
- Transaksjonshistorikk
- Verdiutvikling mot egen kjøpspris

I min samling skal hente fra:

- `ct_v_object_user_state_resolved`
- collection/user-tabeller
- private transaction-tabeller
- private image/document-tabeller
- share/access-tabeller

I min samling skal ikke bruke `ct_v_object_relations_resolved` som hovedkilde. Relasjonsviewen kan brukes til å vise objektets offentlige relasjoner, men private eierdata må komme fra bruker-/samlingstabeller.

4. Skillet mellom katalogobjekt og samlerobjekt

Et sentralt prinsipp i chatten er skillet mellom katalogobjekt og samlerobjekt.

4.1 Katalogobjekt

Katalogobjektet er hovedobjektet i Collectium-katalogen.

Det identifiseres med:

- `source_key`
- `object_group`
- `object_id`

Eksempel:

- `source_key = norske_sedler`
- `object_group = banknote`
- `object_id = 1`

Katalogobjektet inneholder objektets felles, kontrollerte kataloginformasjon.

4.2 Samlerobjekt

Samlerobjektet er brukerens konkrete eksemplar.

Det kan ha:

- egen kjøpspris
- egen kvalitet
- egen gradering
- egne bilder
- egne notater
- privat proveniens
- egen plassering
- egen transaksjonshistorikk

Derfor skal ikke Samler-fanen automatisk bli det samme som I min samling. Samler viser katalogisk og samlerrelevant informasjon, mens I min samling viser brukerens private eksemplardata.

5. Neon-kontroll: objektpresentasjon

I chatten ble det undersøkt hva Neon har definert for objektpresentasjon.

Objektpresentasjon er definert som en kontrollert UI/UX 8.6-side:

- `page_key: object.presentation`
- `route_path: /objekt/[sourceKey]/[objectGroup]/[objectId]`
- `page_type: object`
- `module_key: object`
- `collectium_standard: ui86_neon`
- `objektnøkkel: source_key + object_group + object_id`

Det betyr at objektpresentasjon ikke skal bygges som en løs side. Den skal være del av DB-/feature-/API-kjeden.

6. Feature keys for objektpresentasjon

Følgende feature keys ble identifisert som relevante:

- `catalog.object.open`
- `object.presentation.view`
- `object.relations.view`
- `object.market.view`
- `object.user_state.view`

Kobling mot segmentene:

Segment	Feature
Åpne objekt	<code>catalog.object.open</code>
Objektinfo/base	<code>object.presentation.view</code>
Historie/relasjoner	<code>object.relations.view</code>
Finans/market	<code>object.market.view</code>
I min samling/brukerstatus	<code>object.user_state.view</code>

Dette gir en tydelig modulstruktur for UI/UX 8.6.

7. API-ruter og views

Relevante API-ruter:

- GET /objekt/[sourceKey]/[objectGroup]/[objectId]
- GET /api/object/presentation
- GET /api/object/relations
- GET /api/object/market
- GET /api/object/user-state

Relevante views:

- ct_v_object_presentation_resolved
- ct_v_no_banknote_object_presentation
- ct_v_object_relations_resolved
- ct_v_object_market_resolved
- ct_v_object_user_state_resolved

Frontend skal lese via API. API skal lese fra resolved views. Frontend skal ikke lese direkte fra rå katalogtabeller som om de var presentasjonskilder.

8. Neon-kontroll: ct_v_object_relations_resolved

I chatten ble ct_v_object_relations_resolved undersøkt grundig.

Først ga SQL feil fordi spørringen brukte kolonner som ikke finnes:

- relation_key
- display_name_no
- period_label_no
- country_scope

Deretter ble view-strukturen kontrollert med information_schema.columns.

Faktiske kolonner i ct_v_object_relations_resolved:

- source_key
- object_group
- object_id
- object_sort_number
- title_no
- relation_type
- relation_label_no
- relation_slug
- relation_href
- relation_source
- relation_payload_json
- resolved_at

Dette betyr at viewen er en objekt-til-relasjon-view.

Den skal brukes til:

- relasjonschips
- klikkbare felt
- Historie-fanen
- Se relasjon
- relasjonsliste i objektpresentasjon
- katalogkortets relasjonsknapper
- navigasjon til relasjonspresentasjon

Den skal ikke alene brukes som full historisk kunnskapsdatabase.

9. Relasjonstyper funnet i Neon

Kontrollen viste at følgende 8 relasjonstyper finnes for Norske sedler:

- ar
- kilde
- person
- publiseringsar
- regent
- utgave
- valor
- variant

Hver type hadde 2 125 relasjoner.

Det betyr:

- 2 125 objekter i Norske sedler
- 8 relasjoner per objekt
- totalt 17 000 relasjonskoblinger

Dette er et sterkt OK-funn for relasjonskoblingen.

10. Relasjoner per objekt

Kontrollen viste at hvert objekt får 8 relasjoner.

Eksempel:

Objekt:

- 1 krone • 1917 • 1 • Blank

Relasjoner:

- ar: 1917
- kilde: norske-sedler
- person: s-cederholm
- publiseringsar: 1917
- regent: haakon-vii
- utgave: skillemyntseddel-type-1-1918-25
- valor: 1-krone
- variant: seddel-type-1

Dette bekrefter at objektets historiske koblingslag er konsistent.

11. URL- og slug-kontroll

Det ble også kontrollert at:

- relation_type
- relation_slug
- relation_href

stemmer med hverandre.

Eksempel:

- relation_type = regent
- relation_slug = haakon-vii
- relation_href = /relasjon/regent/haakon-vii

Resultatet viste href_status = OK i kontrollutdraget.

Låst regel:

Frontend skal bruke relation_href direkte når feltet finnes. Frontend skal ikke bygge URL-er selv.

12. Hva dette betyr for Historie

Historie kan nå bruke ct_v_object_relations_resolved som sikker kobling til relasjonssider.

Historie kan vise klikkbare relasjoner for:

- År
- Publiseringsår
- Kilde
- Person/signatur
- Regent

- Utgave
- Valør
- Variant

Men Historie må hente dypere innhold fra egne detail views når brukeren åpner relasjonspresentasjonen.

Eksempel:

Klikk på:

- `/relasjon/regent/haakon-vii`

skal ikke bare vise at Haakon VII finnes som slug. Relasjonssiden må vise:

- biografi
- regentperiode
- dynasti/kongehus
- historisk kontekst
- relaterte personer
- relaterte objekter
- relevante hendelser
- eventuelle finans-/periodekoblinger

13. Hva dette betyr for Samler

Samler kan bruke samme relasjonskobling som ekstra klikkbare felt, men Samler er ikke primært historisk dybde.

Samler skal bruke:

- `ct_v_object_presentation_resolved`
- `ct_v_no_banknote_object_presentation`
- `ct_v_object_user_state_resolved`

Samler skal vise katalogidentitet og samlerrelevante data, for eksempel:

- tittel
- katalognummer
- valør
- år
- litra
- utgave
- variant
- signatur
- regent
- sjeldenhet
- kvalitet/katalogvurdering
- hjerte/stjerne

- legg i samling
-

14. Hva dette betyr for Finans

Finans er teknisk definert, men datagrunnlaget er foreløpig svakere enn Historie.

Kontrollen viste at markedsdata ofte er null:

- `market_value_raw_no = null`
- `value_raw_no = null`
- `trend_raw_no = null`
- `auction_status_raw_no = null`
- `shop_status_raw_no = null`
- `finance_json = {"grade_values": {}}`
- `market_value_status_no = Mangler markedsverdi`

Derfor skal Finans UI vise manglende data korrekt.

Låst visning ved manglende finansdata:

- Mangler markedsverdi
- Ikke beregnet trend
- Ikke registrert på auksjon
- Ikke registrert i nettbutikk
- Ingen grade-verdier registrert

Ikke vis `0 kr` med mindre Neon/API eksplisitt sier at dette er reell verdi.

15. Hva dette betyr for I min samling

I min samling er ikke ferdig bevist på samme måte som relasjonskoblingen, fordi den krever innlogget bruker og `user_id`.

Men strukturen er tydelig:

I min samling må bruke:

- `ct_v_object_user_state_resolved`
- collection-tabeller
- transaction-tabeller
- private image/document-tabeller
- private provenance-tabeller
- share/access-tabeller

I min samling må være tilgangsstyrt.

Private data skal ikke lekke til:

- andre brukere
- forhandlere
- selgere
- offentlig frontend
- relasjonssider
- generisk katalog

Kun innlogget eier, eller godkjent delt tilgang, skal se private samlingsdata.

16. UI/UX 8.6-visningsmodell

UI/UX 8.6 bør bruke denne modellen:

Visningskort

Kortet viser kompakt:

- bilde
- tittel
- valør
- år
- utgave
- variant
- sjeldenhet
- pris/status hvis finnes
- trend hvis finnes
- hjerte
- stjerne
- åpne objekt
- se relasjon

Kortet skal ikke vise alt. Kortet skal lede videre til objektpresentasjon og relasjoner.

Objektpresentasjon

Objektpresentasjon er hovedsiden for ett objekt.

Den bør ha bryterne:

- Objektinfo / Base
- Samler
- Historie
- Finans
- I min samling
- Relasjoner

Objektpresentasjon skal hente data fra API og resolved views.

Relasjonspresentasjon

Relasjonspresentasjon er egen side:

- `/relasjon/[type]/[slug]`

Eksempler:

- `/relasjon/regent/haakon-vii`
- `/relasjon/person/s-cederholm`
- `/relasjon/ar/1917`
- `/relasjon/utgave/skillemynstseddel-type-1-1918-25`
- `/relasjon/valor/1-krone`
- `/relasjon/variant/seddel-type-1`
- `/relasjon/kilde/norske-sedler`

Relasjonspresentasjon skal vise kunnskapsnode, ikke bare en liste med objekter.

17. Segmentregel for UI/UX 8.6

Låst regel:

Samler, Historie og Finans skal være faste informasjonssegmenter på visningskort, objektpresentasjon og relasjonspresentasjon.

I min samling skal være en egen bruker-/eierflate for det konkrete eksemplaret.

Samler viser katalogisk og samlerrelevant informasjon.

Historie viser historisk og relasjonell kontekst.

Finans viser verdi, marked og økonomisk kontekst.

I min samling viser privat brukerdata og brukerens konkrete eksemplar.

18. Dataflyt

Anbefalt dataflyt:

```
React / Next.js frontend
→ API route
→ Neon resolved view
→ JSON response
→ UI/UX 8.6 segment
```

Eksempel:

Objektpresentasjon

- /api/object/presentation
- ct_v_object_presentation_resolved
- base objektinfo

Historie

- /api/object/relations
- ct_v_object_relations_resolved
- relation chips / relation_href

Finans

- /api/object/market
- ct_v_object_market_resolved
- verdi / trend / markedssstatus

I min samling

- /api/object/user-state
- ct_v_object_user_state_resolved
- brukerstatus / samlingsstatus

19. Viktige SQL-kontroller fra chatten

19.1 Kontroll av relasjonstyper

```
select
  relation_type,
  count(*) as relation_count
from ct_v_object_relations_resolved
where source_key = 'norske_sedler'
  and object_group = 'banknote'
group by relation_type
order by relation_type;
```

Forventet resultat:

ar	2125
kilde	2125
person	2125
publiseringsar	2125

regent	2125
utgave	2125
valor	2125
variant	2125

19.2 Kontroll av relasjoner per objekt

```
select
  object_id,
  title_no,
  count(*) as relation_count,
  string_agg(relation_type || ': ' || relation_slug, ' | ' order by
relation_type, relation_slug) as relations
from ct_v_object_relations_resolved
where source_key = 'norske_sedler'
  and object_group = 'banknote'
group by object_id, title_no, object_sort_number
order by object_sort_number
limit 100;
```

Forventet:

```
relation_count = 8 per objekt
```

19.3 Kontroll av manglende relasjoner

```
with expected as (
  select unnest(array[
    'ar',
    'kilde',
    'person',
    'publiseringsar',
    'regent',
    'utgave',
    'valor',
    'variant'
  ]) as relation_type
),
objects as (
  select distinct
    source_key,
    object_group,
    object_id,
    title_no
  from ct_v_object_relations_resolved
  where source_key = 'norske_sedler'
    and object_group = 'banknote'
```



```

),
missing as (
  select
    o.object_id,
    o.title_no,
    e.relation_type
  from objects o
  cross join expected e
  left join ct_v_object_relations_resolved r
    on r.source_key = o.source_key
   and r.object_group = o.object_group
   and r.object_id = o.object_id
   and r.relation_type = e.relation_type
  where r.object_id is null
)
select *
from missing
order by object_id, relation_type;

```

Forventet:

0 rader

19.4 Kontroll av duplikate relasjoner

```

select
  object_id,
  title_no,
  relation_type,
  count(*) as duplicate_count
from ct_v_object_relations_resolved
where source_key = 'norske_sedler'
   and object_group = 'banknote'
group by object_id, title_no, relation_type
having count(*) > 1
order by duplicate_count desc, object_id, relation_type;

```

Forventet:

0 rader

19.5 Kontroll av URL-avvik

```

select
  object_id,
  title_no,

```

```

relation_type,
relation_slug,
relation_href,
split_part(relation_href, '/', 3) as href_type,
split_part(relation_href, '/', 4) as href_slug
from ct_v_object_relations_resolved
where source_key = 'norske_sedler'
and object_group = 'banknote'
and (
    relation_type is distinct from split_part(relation_href, '/', 3)
    or relation_slug is distinct from split_part(relation_href, '/', 4)
)
order by object_sort_number, relation_type;

```

Forventet:

0 rader

20. Status: hva er OK og hva mangler

OK

- Objektpresentasjon er definert som UI/UX 8.6-side.
- Objektnøkkel er `source_key + object_group + object_id`.
- Relasjonsviewen har riktig faktisk kolonnemodell.
- 2 125 Norske sedler-objekter har 8 relasjoner hver.
- Relasjonstypene er konsistente.
- `relation_href` finnes og skal brukes av frontend.
- URL-/slug-kontroll viser OK i kontrollutdrag.
- Historie har nå et sterkt koblingsgrunnlag.

Delvis OK

- Samler har godt kataloggrunnlag, men må fortsatt kobles tydelig mot brukerstatus.
- I min samling er strukturmessig definert, men må testes med innlogget bruker.
- Finans er teknisk definert, men mangler reelle markedsverdier og trenddata.

Mangler videre

- Relasjonspresentasjonens detail views.
- Biografi for regenter/personer.
- Dynasti/kongehus/maktstruktur.
- Historiske hendelser koblet til år og periode.
- Funn/proveniens som relasjon.
- Finanshistorisk kontekst.
- Reelle markedsverdier.

- Grade values.
 - Auksjonsdata.
 - Nettbutikkdata.
 - Brukerens private samlingsdata testet med `user_id`.
-

21. Anbefalt neste steg

Steg 1: Lås relasjonskoblingen som OK

`ct_v_object_relations_resolved` bør markeres som OK for:

- Norske sedler
- banknote
- objekt til relasjonslenker
- UI/UX 8.6 Historie-bryter
- relasjonschips
- "Se relasjon"

Steg 2: Bygg relation detail views

Det må bygges eller kontrolleres egne views for:

- regent
- person/signatur
- år
- publiseringsår
- utgave
- valør
- variant
- kilde
- periode

Disse skal brukes av `/relasjon/[type]/[slug]`.

Steg 3: Test I min samling med user_id

Det må kjøres test med innlogget bruker for:

- `ct_v_object_user_state_resolved`
- hjerte
- stjerne
- i samling
- kjøpsdata
- private bilder
- kvittering
- notater
- privat proveniens

Steg 4: Bygg Finans-data

Finans trenger datagrunnlag for:

- markedsverdi
- trend
- prisobservasjoner
- grade values
- auksjonsstatus
- nettbutikkstatus
- likviditet
- index-kobling

22. Låst hovedregel

Samler, Historie, Finans og I min samling er fire ulike UI/UX 8.6-brytere med forskjellige datakilder og forskjellig ansvar.

Samler viser katalogisk og samlerrelevant objektinformasjon.

Historie viser objektets relasjoner og historiske kontekst, og bruker `ct_v_object_relations_resolved` som koblingslag.

Finans viser markedsverdi, trend og økonomisk kontekst, men skal aldri vise null som reell verdi.

I min samling viser brukerens private konkrete eksemplar og må være tilgangsstyrt.

`ct_v_object_relations_resolved` er godkjent som objekt-til-relasjon-kobling for Norske sedler fordi alle 2 125 objekter har 8 faste relasjoner: ar, kilde, person, publiseringsar, regent, utgave, valor og variant.

Frontend skal bruke `relation_href` som ferdig URL og skal ikke bygge relasjonslenker selv når `relation_href` finnes.

23. Kort konklusjon

Denne chatten har avklart at UI/UX 8.6-bryterne må behandles som datastyrte segmenter, ikke som løse faner.

Det viktigste tekniske funnet er at Neon nå har et fungerende relasjonskoblingslag for Norske sedler:

- 2 125 objekter
- 8 faste relasjoner per objekt

- 17 000 relasjonskoblinger
- ferdige `relation_href`
- OK slug/href-struktur

Dermed er Historie-bryteren mye sterkere definert enn før.

Neste arbeid bør flyttes fra objekt-til-relasjonskobling til relasjonspresentasjonens innhold: biografi, periode, hendelser, dynasti, proveniens, funn og finanshistorisk kontekst.